



Project Summary: Molecular Graph Valence Audits

What Did We Build?

We built a **coding challenge** — think of it like an escape room, but for programmers.

We created a small software project that *looks* like it should work, but is secretly broken in **8 different ways**. A solver (an AI agent or a human developer) has to find and fix all the bugs to make it pass a 29-check automated test.

The Storyline

A pharmaceutical company's automated pipeline for checking the safety of chemical compounds has broken. Chemists can't release their quality report until a developer fixes the code. The developer has to debug the Python analyzer, fix the build system, and get everything passing — fast.

How Does It Work?

The "Broken" Code (what the solver sees)

workspace/	
├─ molops/	← Python code that checks molecules (has bugs)
├─ fixtures/	← Data files describing 5 molecules (correct)
├─ Makefile	← Build commands to run everything (has bugs)
└─ run_audit.py	← Main script (correct)

The code reads molecular structures (water, methane, benzene, etc.) and checks:

"Does each atom have the right number of bonds?"

For example:

- **Carbon** should always have **4 bonds** (like in methane CH₄)
- **Oxygen** should always have **2 bonds** (like in water H₂O)
- **Hydrogen** should always have **1 bond**

If any atom has too many or too few bonds, the molecule is flagged as invalid.

The Bugs We Hid

BUG	PLAIN ENGLISH ANALOGY
Wrong filename in Makefile	Like telling someone to open <code>report_final_v2.docx</code> when the file is actually called <code>report.docx</code>
<code>make clean</code> deletes input data	Like a "Clear History" button that also deletes your bookmarks
Oxygen valence set to 3	Like saying humans have 6 fingers — the reference table is simply wrong
Only counting bonds in one direction	Like counting only outgoing calls on your phone, ignoring incoming ones
Using neighbor count instead of bond strength	Like counting how many friends you have vs. how <i>close</i> you are to each — a double bond counts as 2, not 1
Aromatic bonds all set to single	Like converting all bold text to regular — loses important information
Non-deterministic report ordering	Like a receipt that prints items in a random order each time
Missing <code>.PHONY</code> declaration	A subtle Makefile issue that can cause "nothing to do" errors

The Auto-Grader

`verify.sh` runs **29 automated checks** across **7 phases**. It's like a teacher's answer key — the solver doesn't get to see it, they just get a pass/fail grade.

Why Is This Useful?

1. Benchmarking AI Coding Agents 🤖

This is the primary use case. Companies building AI coding assistants (like GitHub Copilot, Cursor, Devin, etc.) need standardized tasks to measure:

- *Can the AI find bugs by reading code?*
- *Can it understand chemistry domain rules?*
- *Can it fix a Makefile?*
- *Can it trace a chain of bugs where fixing one reveals another?*

Our task is particularly good for this because the bugs **interact** — fixing Bug A reveals Bug B, which requires understanding Bug C. This tests whether an AI can reason through a multi-step debugging process, not just pattern-match.

2. Developer Interview / Assessment 📄

Companies hiring software engineers could use this as a take-home coding challenge. It tests:

- **Reading existing code** (not just writing new code)
- **Build system literacy** (Makefile debugging)
- **Domain knowledge** (basic chemistry rules)
- **Systematic debugging** (following the chain)
- **Attention to detail** (the non-determinism bug is subtle)

3. Teaching & Training 🎓

Great for CS courses on:

- **Software debugging methodology** — how to approach a broken codebase
- **Build systems** — understanding Make, `.PHONY`, targets, recipes
- **Testing & verification** — how automated test suites work
- **Scientific computing** — intro to cheminformatics concepts

4. CI/CD Pipeline Testing ⚙️

The pattern we used (broken code + verifier script) is exactly how real-world CI/CD works. This project demonstrates:

- How to write comprehensive bash test scripts
 - How to test for determinism (same input → same output)
 - How to test for safety (destructive operations caught)
-

What Are the Real-World Applications?

The Task Format (Debugging Challenges)

APPLICATION	WHO USES IT	EXAMPLE
AI agent benchmarks	Google, OpenAI, Anthropic	SWE-bench, HumanEval
Coding interviews	Tech companies	HackerRank, LeetCode
CTF competitions	Security researchers	Capture-the-flag events
Education platforms	Universities	Auto-graded assignments

The Domain (Molecular Graph Validation)

The actual *chemistry* in this project is a simplified version of what real pharmaceutical software does:

OUR PROJECT	REAL-WORLD EQUIVALENT
JSON molecule files	Chemical databases (PubChem, ChEMBL)
Valence checking	Drug candidate screening
Bond-order normalization	Molecular descriptor computation
Audit reports	Regulatory compliance reports
Make-driven pipeline	Jenkins/Airflow pipelines at pharma companies

Real tools like **RDKit**, **Open Babel**, and **Schrödinger** do exactly this — validate molecular structures before expensive lab work begins. Our project is a toy version of that pipeline.

At a Glance

WHAT WE BUILT

A debugging puzzle with 8 hidden bugs in a molecular chemistry audit tool

TECH STACK

Python + Makefile + Bash verifier • No internet, no dependencies

SCALE

15 files | 8 bugs | 29 checks | 5
molecules

DIFFICULTY

Expert: ~30 min | Junior: ~90
min

PRIMARY USE

Benchmark AI coding agents & test developer debugging skill